WEEK 2 OF 4

Motion Becomes Number

The day the software starts seeing you

By the end of the hour your fingertip will be three live numbers.

Everything after this week is just deciding what those numbers should control.

Where we are

Yesterday was images. Today we add numbers.

WEEK 1 gave you

- Tab to make a node, drag to wire it, corner square to see it
- A chain of TOPs turning your webcam into something else
- Named Nulls at section boundaries
- "Address, not a wire" — connections made in a parameter field

Today adds

CHOPs — streams of numbers instead of pictures.

MediaPipe, which turns your hand into those numbers.

The Math CHOP, for rescaling numbers into whatever range you actually need.

The hour

Open your Week 1 file before we start. We are adding to it, not starting again. If yesterday's file is broken or missing, say so now and I will send you a fresh one.

CHOP — Channel Operator

last week's nodes carried pictures. These carry numbers.

A channel is one number that keeps changing. A CHOP carries a bundle of them.

1 **It updates constantly**

Sixty times a second, same as everything else. A CHOP is not a stored list — it is a live reading.

2 **Channels have names**

Your hand does not arrive as one number. It arrives as dozens, each named. Most of your work is picking the ones you want.

3 **Its viewer is a graph**

Click a CHOP's corner square and you see a moving line. Flat line means nothing is coming through — your first debugging check.

What MediaPipe actually gives you

It watches your webcam and reports where your hands are.

21 points per hand

Wrist, knuckles, and the joints and tip of every finger. Each reports a position that updates live. Two hands tracked at once.

Joint names: MCP is the big knuckle at the palm, PIP the middle, DIP nearest the tip, TIP the end. We want TIP.

The good news

This plugin hands you the landmarks already named, as ordinary CHOP channels. You do not have to decode anything.

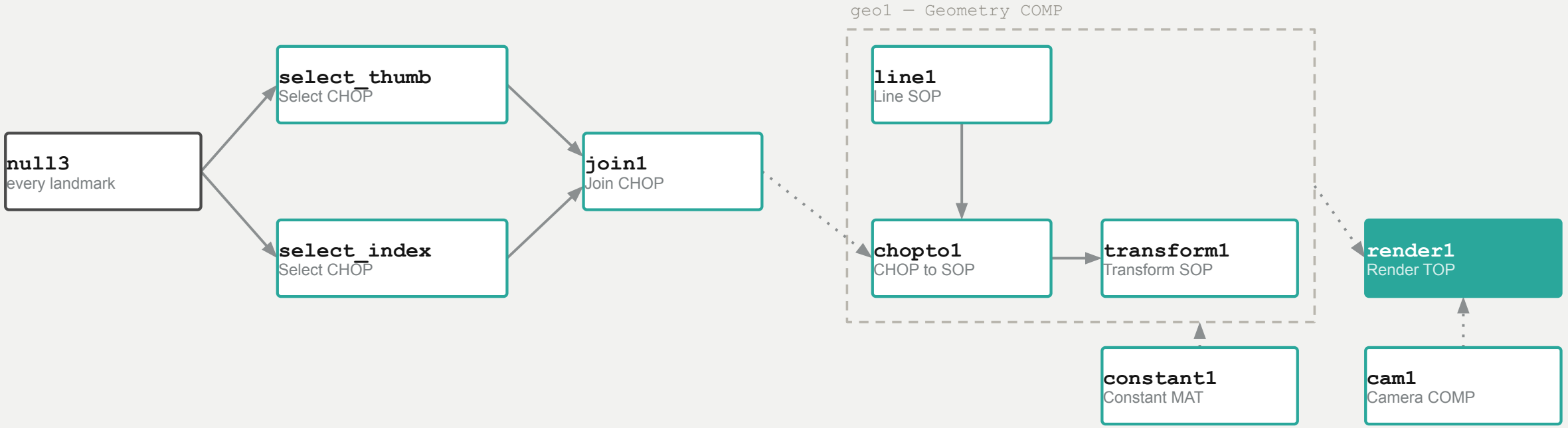
Find the hand_tracking component's normalized_data output. Everything you need is in there.

It is just numbers

There is no magic in the tracking. Your hand becomes a set of coordinates, and coordinates are things you can add, scale, smooth and compare. Once you accept that, gesture control stops being mysterious and becomes arithmetic.

Drawing between thumb and fingertip

From two sets of numbers to a line on screen.



→ a wire you drag ⋯► an address — drop the node onto a parameter field: cam1 into Camera, geo1 into Geometry, constant1 into Material

`select_thumb / select_index` one landmark each. Rename To t* so both read tx ty tz — Join only merges matching names.

`join1` stacks them into 2 samples. Sample 0 is the thumb, sample 1 is the fingertip.

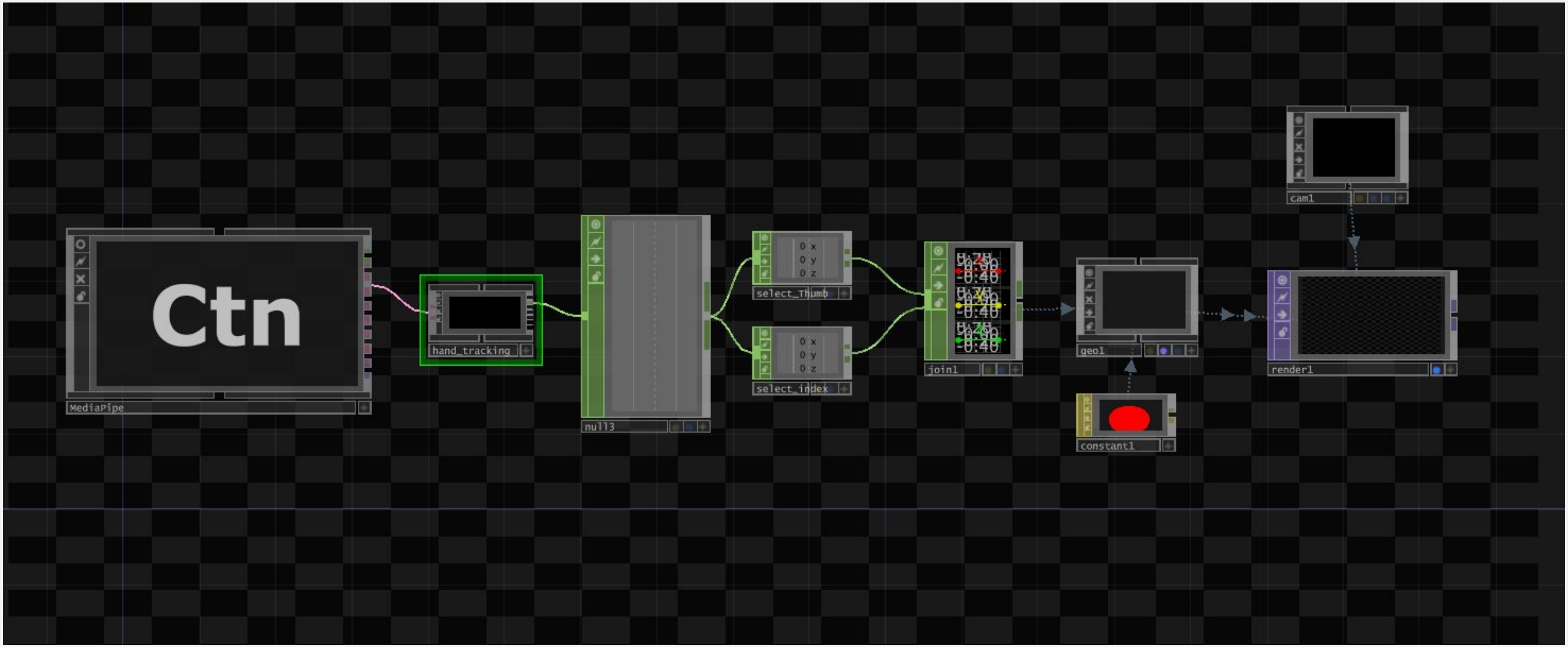
`line1 → chopto1` a blank 2-point line. chopto1's CHOP field reads ../join1 and drops the points onto the samples.

`transform1` remaps 0–1 into SOP space. Scale 2, -2, 1 · Translate -1, 1, 0.

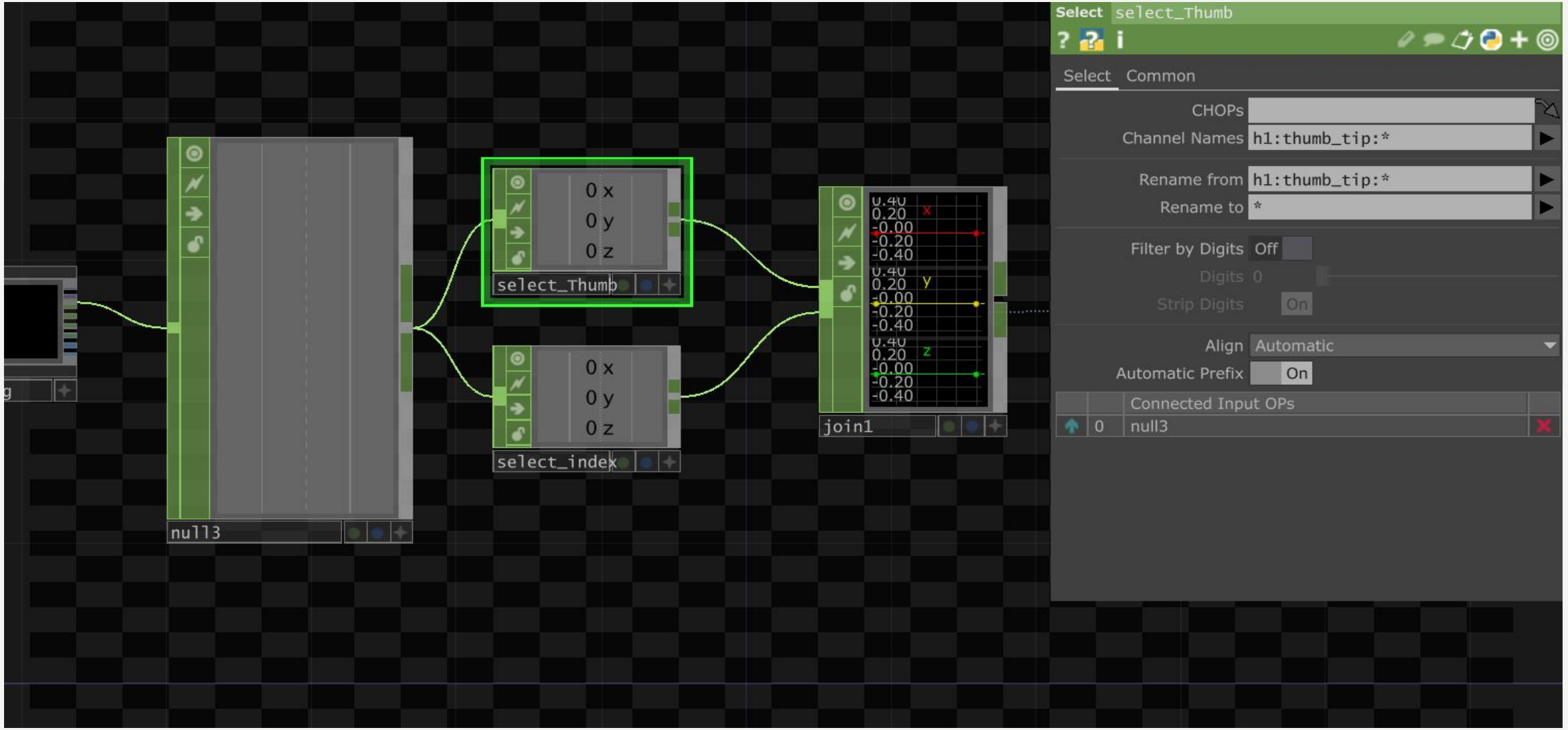
Two flags, two levels. Display feeds the viewers, Render feeds the Render TOP. Both transform1 and geo1 need their Render flag on — one without the other gives you an empty frame and no warning. Seeing it in a viewer but not in render1 is always this.

Drawing between thumb and fingertip

From two sets of numbers to a line on screen.



Reading the channel names



Two things that will confuse you

Both are real behaviours, not bugs. Knowing them now saves an hour later.

h1 is not "left"

h1 is simply the FIRST hand the camera noticed. Drop that hand and raise the other, and h1 now refers to a different physical hand.

h2 is not "right" either

Same reason. If you need to know which hand is genuinely which, you must read the handedness value, not the h1/h2 slot.

The image is mirrored

Your webcam feed is flipped before MediaPipe sees it, the way a bathroom mirror flips you. This is deliberate — it makes moving feel natural.

So handedness reads "backwards"

Raise your actual right hand and the label says Left. It is describing the mirrored picture, which matches what YOU see on screen. Correct, just surprising.

For this week, treat them simply as "first hand" and "second hand." That is honest, and for driving two particle emitters it makes no practical difference which is which.

The Math CHOP — getting numbers into the right range

The tracking counts in one set of units. Whatever you are driving expects a different set.

Nobody translates for you. This is the single most common reason a connection 'does nothing' or sends something flying off screen.

How to do it properly

- Watch the real numbers in the viewer as you move your hand. Note the lowest and highest you actually reach.
- Put those in the Math CHOP's From Range, and what the target needs in the To Range.

Two fixes that look like bugs

Moving right sends it left? Flip the sign — the mirrored image again.

Moving up sends it down? Screen coordinates often count downward. Same fix.

The method, again: when something misbehaves, click through the chain and read the numbers at each step. Where they stop making sense is where to fix. Same approach as last week's resolution problem — different symptom, identical method.

If the tracking will not work

Nobody sits out this session.

No numbers at all

MediaPipe has not finished starting, or another app has the camera. Quit everything else, restart it, give it a full minute.

It was working, now frozen

Check Cooking. Right-click the component — if Cooking got switched off, it freezes on its last frame and looks exactly like a loading screen. Easy to click by accident.

Numbers freeze halfway

The tracking lost your hand. Better light, plain background, hand fully in frame, palm toward the camera.

Channel names look different

Plugin versions vary. Read the names in YOUR channel list and use those — the ones on my slides are what our build produces, not a universal standard.

Everything downstream of null_tip_h1 neither knows nor cares where the numbers came from. Real tracking, a recording, or a slider you drag by hand — identical behaviour. That is the Null habit earning its keep.

Homework: pinch to prove it

Twenty minutes. One new idea, built on everything from today.

Make a number that gets smaller when you pinch your thumb and finger together.

1

Add the thumb

Back to your Select CHOP. Add the thumb tip's channels alongside the index finger's.

2

Measure the gap

The distance between two points. Close together, small number.

3

Show it working

Drive anything with it. A circle's size, a blur amount, the brightness of last week's webcam effect. Your choice.

Post a short screen recording, not a screenshot.

Movement is the whole point and a still image cannot show it. Ten seconds is plenty. Stuck? Post where you got to and I will unstick you before tomorrow.

Next Week:

Day 3 — Particles

A deliberate break from hand tracking. We build a system of thousands of moving points, learn how anything 3D reaches your screen, and drive it with the mouse. On Day 4 your hands take over.

Before then

- [ ] Save your file. Same file all week.
- [ ] Post your ten-second pinch recording.
- [ ] Nothing new to install — next week uses what you already have.
- [ ] Close anything using your camera before we start.